# DAT120 og DAT125 terminologi tema 1: Introduksjon

## Tallsystemer

| Norsk term | Engelsk term | Forklaring |
|---|---|---|
| Base til et tallsystem | Base of a number system | Basen til et posisjonsbasert tallsystem er hvor mange ulike talltegn det er og det største tallet som kan uttrykkes med ett siffer. I det vanlige titallssystemet (desimalt tallsystem) er det ti tegn: 0, 1, 2, 3, 4, 5, 6, 7, 8 og 9. |
| Bit | Bit, binary digit | En verdi som kan være enten 0 eller 1. Tilsvarer en bryter som kan være enten av eller på. Den minste enheten av informasjon. |
| Binært tallsystem | Binary number system | Et posisjonsbasert tallsystem med base 2, med bare sifrene 0 og 1. Så for eksempel 101 er $1*2^2 + 0*2^1 + 1*2^0 = 4 + 0 + 1 = 5$. |
| Byte | Byte | Gruppe med 8 bits. Grunnenheten i minnet til en datamaskin. Kapasiteten til et minne måles gjerne i større enheter:<br>- Kilobyte: 1000 bytes<br>- Megabytes: 1 million bytes (1 000 000)<br>- Gigabytes: 1 milliard bytes (1 000 000 000)<br>- Terabytes: 1 billion bytes (1 000 000 000 000) |
| Desimalt tallsystem | Decimal number system | Et posisjonsbasert tallsystem med base 10, med de normale sifrene 0-9. Så for eksempel 101 er $1*10^2 + 0*10^1 + 1*10^0 = 100 + 0 + 1$. |
| Heksadesimalt tallsystem | Hexadecimal number system | Et posisjonsbasert tallsystem med base 16, med sifrene 0-9 samt A-F hvor A=10, B=11 og så videre til F=15. Dette tallsystemet er mye brukt i dataverdenen siden et heksadesimalt siffer kan uttrykkes med 4 bits. En byte kan uttrykkes med to heksadesimale sifre. Så for eksempel 101 er $1*16^2 + 0*16^1 + 1*16^0 = 256 + 0 + 1 = 257$. |
| Posisjonsbasert tallsystem | | Tallsystem der store tall uttrykkes med de samme sifrene som små tall, men i en annen posisjon. For eksempel det vanlige tallsystemet i dag hvor tallet 1752 er ett tusen, sju hundre og femti to. De samme sifrene brukes for tusener og enere, bare med ulik posisjon. Et eksempel på et tallsystem som bare i begrenset grad er posisjonsbasert er romertallene. |

## Deler av en datamaskin

| Norsk term | Engelsk term | Forklaring |
|---|---|---|
| Arbeidsminne, hovedlager, RAM | RAM, Random Access Memory, main memory | Arbeidsminnet til en datamaskin. Dette minnet er raskt, men flyktig. Innholdet forsvinner når man slår av maskinen eller strømmen går. |
| Harddisk | Hard disk, hard drive | Et varig datalager som er innebygd i datamaskina. Tregere enn arbeidsminnet. Både harddisker og eksterne lager som minnepenner er organisert i filer og mapper. Termen kommer av at man tidligere brukte en hard plate (disk) til dette i motsetning til de mykere platene (engelsk floppy disk) man brukte for disker man kunne sette inn og ta ut av maskina. Men et innebygd varig datalager kalles ofte en harddisk selv om den er basert på en annen teknologi enn dette. |
| Prosessor | Processor, Central Processing Unit, CPU | Enheten som utfører arbeidet i en datamaskin. I hver klokkesyklus så leser den en instruksjon i maskinkode og utfører den. I neste klokkesyklus leser den neste instruksjon, på neste minneadresse. |

## Maskinvare og programvare

| Norsk term | Engelsk term | Forklaring |
|---|---|---|
| Assemblerspråk | Assembly | Et språk hvor man gir hver enkelt maskinkodeinstruksjon et navn som er mer leselig for mennesker enn bitmønstre er. |
| Maskinkode | Machine code | Instruksjoner som prosessoren forstår. Hver instruksjon i maskinkode er et bit-mønster på en eller flere byte som er tilnærmet uleselig for mennesker. Hver instruksjon gjør en svært liten operasjon, så å programmere i maskinkode eller assemblerspråk er svært arbeidskrevende. |
| Maskinvare | Hardware | Den fysiske elektronikken i en datamaskin |
| Operativsystem | Operating system | Et program som styrer datamaskina og som sørger for at de andre programmene som kjører på den ikke går i beina på hverandre. Android, iOS, Linux og Windows er eksempler på operativsystemer. |
| Programmeringsspråk | Programming language | Språk for å programmere en datamaskin og hvor hver instruksjon er en større enhet enn i assemblerspråk slik at det blir mindre jobb å programmere. Dette oversettes til maskinkode av en kompilator eller tolker før det kjøres. Programmeringsspråk er spesialiserte språk hvor du kan uttrykke algoritmer presist nok til at en dum maskin kan utføre dem. |
| Programvare | Software | Programmer som maskinvaren skal kjøre |

## Filer

| Norsk term | Engelsk term | Forklaring |
|---|---|---|
| fil | file | En samling data som hører sammen og typisk er lagret av et enkelt program. Et digitalt bilde eller et Word-dokument er eksempler på filer. |
| Mappe, katalog | Folder, directory | Filer er organisert i mapper. Ei mappe kan inneholde mange filer. Ei mappe kan også inneholde andre mapper. Man kan ha mange nivåer med mapper, den ene inni den andre. |
| Rot-mappe | Root directory | Dette er den øverste mappa, som er starten på mappestrukturen. Dette er den eneste mappa som ikke ligger i en annen mappe. I Windows har hver enhet (harddisk, minnepenn osv.) sin egen rot-mappe. Hver enhet får sin egen bokstav-kode av Windows, harddisken får vanligvis C:\. I andre systemer som iOS og Linux er det en enkelt rotmappe for hele systemet, og minnepenner og liknende blir lagt til som mapper et sted i mappestrukturen. |
| Sti | Path | Hvor du skal gå i mappestrukturen for å finne ei bestemt fil. For eksempel vil stien C:\Users\Erlend\Documents\test.docx (windows) eller /Users/Erlend/Documents/test.docx (mac eller linux) referere til fila «test.docx» som ligger i mappa «Documents». Mappa Documents ligger i mappa «Erlend» som ligger i mappa «Users» som ligger i rotmappa.<br><br>Dette eksemplet er en absolutt sti. Absolutte stier starter i ei rot-mappe. Man kan også ha relative stier. En relativ sti starter i ei mappe og viser hvordan du navigerer deg fra mappa du står i nå og til der fila ligger. Et Python-prosjekt har som regel ei prosjektmappe som er start-mappe for slike relative stier. |

## Programmering og programmeringsspråk

| Norsk term | Engelsk term | Forklaring |
|---|---|---|
| Algoritme | Algorithm | Oppskrift på hvordan å løse en oppgave som er så detaljert og entydig at en dum maskin kan utføre den. Kokebok-oppskrifter er eksempler på algoritmer fra en annen bransje enn databransjen. |
| Datatype | Data type | Informasjon om hvordan en sekvens av bits i minnet skal tolkes. Utdypet nærmere i terminologi for tema 2. |
| Statisk typet | Statically typed | Alle variabler i et statisk typet språk har en definert datatype som defineres når variabelen blir lagd, enten eksplisitt eller gjennom å tilordne variabelen en startverdi. For eksempel blir variabelen lagd med setningen «variabel=5» så ser systemet at 5 er et heltall, så datatypen til variabelen blir «heltall». Eksempler C, C++, C#, Java, Go.<br><br>Fordelen med statisk typete språk er at de kan sjekke om hver tilordning er lovlig når den gjøres. Slik kan de finne feil tidligere, og feilene derfor er lettere å rette. Ulempen er at de er mindre fleksible og at det blir litt mer kode. |
| Dynamisk typet | Dynamically typed | Variabler i språket har ingen definert datatype og kan i prinsippet referere til hva som helst. Dataverdier og objekter har fortsatt en datatype. Eksempler Matlab, Python, Javascript, PHP |
| Kildekode | Source Code | Programkoden til et program. Koden du skriver i øvingene i DAT120 er kildekode. I tolkete språk blir kildekoden kjørt direkte. I kompilerte språk så blir kildekoden kjørt gjennom en kompilator som lager ei kjørbar fil, og så et det denne kjørbare fila du faktisk kjører. |
| Kompilator | Compiler | En kompilator er et program som oversetter et helt program fra et programmeringsspråk til maskinkode. En kompilator er som en oversetter som oversetter ei bok. For å kjøre programmer i et kompilert språk så må man kompilere programmet først. |
| Kompilerte språk | Compiled languages | Et kompilert språk er laget for å oversettes av en kompilator til maskinkode før programmet kjøres. Kompilerte språk er typisk statisk typet, men ikke nødvendigvis. Programmer i kompilerte språk kjører fortere.<br><br>Eksempler: C, C++, og Go. Java er en hybrid mellom kompilert og tolket da Java kode kompileres til Bytecode i stedet for maskinkode. Bytecode tolkes deretter. Bytecode er såpass nært maskinkode at tolkeren er mye enklere og raskere enn en tolker for et språk på høyere nivå. |
| Imperativt programmeringsspråk | Imperative programming language | Et program i et slikt språk er en serie med kommandoer som utføres i rekkefølge, typisk fra starten av fila eller fra et definert startpunkt. De fleste språkene i dette temaet, inkludert C, Java, MATLAB og Python, er imperative |

| | | |
|---|---|---|
| Funksjonelt programmeringsspråk | Functional programming language | Et program i et slikt språk er en serie med funksjonskall. Et viktig prinsipp i funksjonell programmering er at et funksjonskall ikke skal ha noen sideeffekter. Funksjonen skal ta inn ei liste med parametere og returnere en eller flere nye verdier, men skal ikke endre noen av verdiene den fikk inn eller gjøre andre endringer av tilstanden til systemet. Eksempler på funksjonelle språk: LISP, Clojure, F#. |
| Objektorientert programmeringsspråk | Object Oriented programming language | Språket bruker konsepter fra objektorientert programmering (temaet klasser og objekter i DAT120, ikke pensum i DAT125). Et objektorientert språk er også medlem av en av de andre familiene. C++, C#, Java og Python er objektorienterte mens C og MATLAB ikke er det. |
| Tolker | Interpreter | I programmering så er en tolker et program som leser kildekode linje for linje, oversetter linja til maskinkode, og kjører maskinkoden for linja. I dette faget så blir Python tolkeren som følger med Anaconda brukt. En tolker er som en døvetolk eller simultanoversetter som oversetter en tale mens talen holdes. |
| Tolkete språk | Interpreted languages | Et tolket språk er laget for at hver linje skal tolkes for seg. Man kjører programmet gjennom å kjøre en tolker. Man trenger ikke å kompilere kode før man kjører.<br><br>Tolkete språk er typisk dynamisk typet, men ikke nødvendigvis. Tolkete språk er ofte mer fleksible, men programmer i tolkete språk kjører tregere enn programmer i kompilerte språk.<br><br>Eksempler: Matlab, Python |
| Variabel | Variable | Navngitt sted i hovedlageret hvor det er lagret en verdi. |

## Basis Python

| Norsk term | Engelsk term | Forklaring |
|---|---|---|
| Distribusjon | Distribution | For programmer med åpen kildekode har man ofte flere distribusjoner. En distribusjon er en installasjonsfil som inneholder en pakke med ting man installerer sammen. For Python har man blant annet standard Python og Anaconda. Standard Python er ofte mer oppdatert med et litt høyere versjonsnummer og derfor noen flere egenskaper, mens Anaconda har med seg en god del nyttige tilleggspakker og tilleggsprogram slik som utviklingsmiljøet Spyder. Disse tilleggspakkene må man eventuelt installere selv separat om man bruker standard Python. |
| Konsoll, Terminal | Console, Terminal | Konsoll kan referere til flere ting:<br>- Vinduet hvor Python programmet skriver ut ting og ber om at brukeren skriver inn ting.<br>- Konsollet til operativsystemet (cmd kommandoen i Windows) hvor man kan skrive kommandoer til operativsystemet. Denne er av og til kalt «Terminal»<br>- Python konsollet / kommandolinja, hvor du kan kjøre Python kommandoer enkeltvis |
| Nøkkelord | Keyword | Et ord som er definert i selve Python-språket. Slike ord kan ikke brukes som navn på variabler eller andre strukturer. Eksempel while, if. Alle utviklingsmiljøer uthever nøkkelord i en spesiell farge. |
| Programutviklingsmiljø, utviklingsmiljø | Integrated Development Environment, IDE | Et program som hjelper deg å programmere og som gir mye støtte under arbeidet med å programmere. Man kan programmere uten et utviklingsmiljø, men det er mer arbeidskrevende. Noen eksempler på utviklingsmiljø for Python: Spyder, PyCharm, Visual Studio Code. |
| Skript | Script | En serie med kommandoer i Python eller et annet tolket språk som blir kjørt etter hverandre automatisk, i motsetning til bruk av Python konsollet for å kjøre enkeltkommandoer. Skript lagres i .py filer. Python programmer er skript. |